

Vysalytica Project Structure Summary

Generated: November 13, 2025

Repository: VysalyticaLab/Vysalytica (Private)

Backend URL: <https://vysalytica-api.onrender.com>

Repository Overview

The Vysalytica repository contains a **Flask-based REST API backend** with multiple frontend options (Streamlit current, Next.js in development).

Key Statistics

- **Total Files:** 175
 - **Primary Language:** Python
 - **Repository Size:** ~102 MB
 - **Last Updated:** November 12, 2025
-

Project Structure

```

vysalytica/
├── api/                                # Main Flask API backend
│   ├── __init__.py                    # Package entry point (exports app)
│   ├── api.py                          # Main Flask application (29KB)
│   ├── engine_crawl.py                 # Web crawling engine
│   ├── engine_parse.py                 # Content parsing engine
│   ├── engine_report.py                # Report generation (MD/DOCX)
│   ├── engine_rules.py                 # Basic rules engine
│   ├── engine_rules_enhanced.py        # Enhanced rules (92KB)
│   └── vysalytica/                     # Core business logic package
│       ├── __init__.py
│       ├── config.py                   # Configuration management
│       ├── middleware.py                # Flask middleware (auth, rate limiting)
│       ├── plans.py                     # Subscription plan logic
│       ├── wsgi.py                      # WSGI entry point
│       └── db/                          # Database layer
│           ├── __init__.py
│           ├── models.py                # SQLAlchemy models
│           ├── migrations.py            # Database migrations
│           └── rule_seed_data.py
│       ├── engine_ai_visibility.py      # AI visibility analysis
│       ├── engine_answer_graph.py       # Answer graph generation
│       ├── engine_fixgen.py             # Fix generation engine
│       └── engine_playbooks.py          # Playbook generation
├── frontend/                           # Next.js frontend (in development)
│   ├── src/
│   │   ├── app/                        # Next.js app directory
│   │   └── components/                 # React components
│   ├── package.json                    # Node dependencies
│   ├── next.config.js
│   ├── tailwind.config.ts              # Tailwind CSS config
│   └── tsconfig.json                    # TypeScript config
├── streamlit_app.py                     # Current Streamlit frontend (20KB)
├── scripts/
│   └── import_check.py                 # Dependency validation
├── archive/                             # Historical/deprecated code
│   ├── ai_visibility_mvp/              # Previous MVP version
│   └── [various docs]                  # Deployment guides, summaries
├── requirements.txt                     # Python dependencies
├── runtime.txt                          # Python version (3.x)
├── Procfile                             # Render deployment config
├── .env.example                         # Environment template
├── README.md                           # API documentation
└── CHANGES_SUMMARY.md                 # Change log

```

Backend Framework: Flask

Core Dependencies

```
flask>=2.3,<3
flask-cors==4.0.0
flask-limiter==3.5.0
gunicorn>=21,<22
sqlalchemy==2.0.36
psycopg[binary]>=3.1.0
```

Web Scraping Stack

```
requests==2.31.0
beautifulsoup4==4.12.2
extract>=0.16.0
lxml>=4.9.3
trafilatura==1.6.2
```

AI/LLM Integrations

```
openai==1.109.1
anthropic==0.72.0
routellm==0.2.0
```

Additional Tools

```
python-docx==1.1.0
pydantic>=2.0
pandas==2.2.3
numpy>=2.0
```

Frontend Options

1. Current: Streamlit (Production)

- **File:** streamlit_app.py (20KB)
- **Status:** Active, deployed
- **Purpose:** Quick prototyping UI

2. In Development: Next.js + React

- **Location:** frontend/ directory
 - **Stack:** Next.js, TypeScript, Tailwind CSS
 - **Status:** Under development
 - **Purpose:** Production-grade web interface
-

Deployment Configuration

Render Deployment

- **Command:** `gunicorn api:app -w 2 -k gthread -b 0.0.0.0:$PORT --timeout 120`
- **Workers:** 2 (gthread mode)
- **Timeout:** 120 seconds
- **Entry Point:** `api:app` (from `api/__init__.py`)

Environment Variables (from `.env.example`)

```
ENV=prod
DATABASE_URL=postgres://USER:PASS@HOST:5432/DBNAME
ROUTE_LLM_API_KEY=
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
CORS_ALLOWED_ORIGINS=https://your-streamlit.streamlit.app,https://vysalytica-land-
ing.vercel.app
QUICKSCAN_PAGE_LIMIT=3
LIMITER_STORAGE_URI=memory
SENTRY_DSN=
APP_VERSION=0.1.0
DEBUG=false
```

Key API Features

Based on the structure and README:

1. **Website AI Visibility Auditing**
 - Crawling engine (`engine_crawl.py`)
 - Content parsing (`engine_parse.py`)
 - AI visibility analysis (`engine_ai_visibility.py`)
 2. **Citation Tracking & Analysis**
 - Answer graph generation (`engine_answer_graph.py`)
 - Rules engine (`engine_rules_enhanced.py`)
 3. **API Key Management & Rate Limiting**
 - Middleware layer (`middleware.py`)
 - Plan enforcement (`plans.py`)
 4. **Report Generation**
 - Markdown/DOCX output (`engine_report.py`)
 - Fix generation (`engine_fixgen.py`)
 - Playbook creation (`engine_playbooks.py`)
 5. **Database Layer**
 - PostgreSQL via SQLAlchemy
 - Models and migrations in `api/vysalytica/db/`
-



Database

- **Type:** PostgreSQL
- **ORM:** SQLAlchemy 2.0.36
- **Driver:** psycopg (binary)
- **Models Location:** `api/vysalytica/db/models.py`
- **Migrations:** `api/vysalytica/db/migrations.py`



API Endpoints

Health check example from README:

```
curl http://localhost:8000/api/health
# Returns: {"status": "healthy", "version": "0.1"}
```

Full endpoint documentation would be in `api/api.py` (29KB file).



Development Workflow

Local Setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env with local config
```

Run Locally

```
gunicorn api:app -w 2 -k gthread -b 0.0.0.0:8000 --timeout 120
```

Access

- API: `http://localhost:8000`
- Health: `http://localhost:8000/api/health`



Related Repositories

From the GitHub search, there are companion repos:

1. **vysalytica-streamlit-ui** (Public)
 - Separate Streamlit UI repo
 - 56 KB, Python

2. **vysalytica-react-ui** (Public)
 - React frontend (empty/new)
 - 0 KB
3. **vysalytica-landing** (Public)
 - Landing page
 - 4 KB, HTML

Architecture Summary

Backend: Flask REST API with PostgreSQL database

Current Frontend: Streamlit (monolithic in main repo)

Future Frontend: Next.js + React (in development)

Deployment: Render (backend), likely Streamlit Cloud (frontend)

AI Stack: OpenAI + Anthropic + RouteLLM for intelligent routing

The project follows a **microservices-ready architecture** with clear separation between:

- API layer (`api.py`)
- Business logic (`vysalytica/` package)
- Data layer (`vysalytica/db/`)
- Processing engines (crawl, parse, rules, report)
- AI engines (visibility, fixgen, playbooks, answer graph)

Next Steps for Migration

To migrate from Streamlit to Next.js:

1. **Complete Next.js frontend** in `frontend/` directory
2. **Ensure API endpoints** are well-documented and stable
3. **Test CORS configuration** for new frontend domain
4. **Deploy Next.js** separately (Vercel/Netlify)
5. **Update CORS_ALLOWED_ORIGINS** to include new frontend URL
6. **Deprecate** `streamlit_app.py` once migration is complete